Nil Galiano, Maria Sannino

API and Dockerization

1. **Start a new project called practice1.**
   You're going to create an API. You can use Flask, FastAPI (recommended), Django, or any other python framework you're familiar with… or any other API framework from other language (Golang? Node.JS?).
   Put it inside a docker.

   First of all we created a new project named practice1 and we also implemented an API using FastAPI since it was the recommended one. In order to implement the API we just follow the instructions from the FastAPI website: https://fastapi.tiangolo.com/#run-it

   Then in order to put it inside a docker, first of all we understand what dockers is used for since we had never used it before looking at: https://docs.docker.com/get-started/

   To check if the container was correctly initialized, we used the following command in the terminal: *curl http://localhost::8000*.

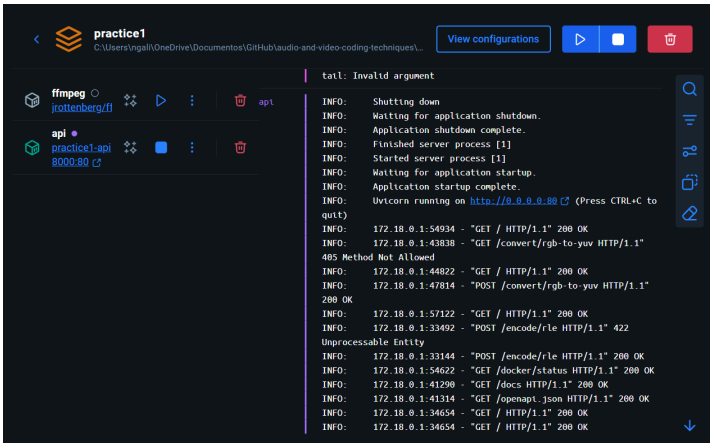
```
PS C:\Users\ngali\OneDrive\Documentos\GitHub\audio-and-video-coding-techniques\practice1> curl http://localhost:8000


StatusCode        : 200
StatusDescription : OK
Content           : {"message":"Image Processing API - Practice 1"}
RawContent        : HTTP/1.1 200 OK
                    Content-Length: 47
                    Content-Type: application/json
                    Date: Wed, 26 Nov 2025 16:16:55 GMT
                    Server: uvicorn

                    {"message":"Image Processing API - Practice 1"}
Forms             : {}
Headers           : {[Content-Length, 47], [Content-Type, application/json], [Date, Wed, 26 Nov 2025 16:16:55 GMT],
                    [Server, uvicorn]}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 47
```

2. **Put ffmpeg inside a Docker.**

   Now to put the ffmpeg inside a Docker we follow the same procedure as we did for the API and we also configure Docker so that this container can be called from the API. Also following the two previous websites.



3. **Include all your previous work inside the new API. Use the help of any AI tool to adapt the code and the unit tests.**

   With the help of AI tools and following the instructions from FastAPI website again: https://fastapi.tiangolo.com/tutorial/body/#use-the-model we were able to model our code from the previous seminar in order that everything works correctly.

4. **Create at least 2 endpoints which will process some actions from the previous S1.**

   We created two endpoints "/convert/rgb-to-yuv" and "/encode/rle" following the steps from the previous links and these ones:
   https://fastapi.tiangolo.com/tutorial/path-params/
   https://docs.python.org/3/library/json.html

   In order to see that both endpoints work correctly we used the following commands in the terminal:

   curl -Method POST http://localhost:8000/convert/rgb-to-yuv `
   -Headers @{ "Content-Type"="application/json" } `
   -Body '{"r":255,"g":0,"b":0}'

```
PS C:\Users\ngali\OneDrive\Documentos\GitHub\audio-and-video-coding-techniques\practice1> curl -Method POST http://local
host:8000/convert/rgb-to-yuv -Headers @{ "Content-Type"="application/json" } -Body '{"r":255,"g":0,"b":0}'


StatusCode        : 200
StatusDescription : OK
Content           : {"y":76.24499999999999,"u":-37.518150000000006,"v":156.825}
RawContent        : HTTP/1.1 200 OK
                    Content-Length: 59
                    Content-Type: application/json
                    Date: Wed, 26 Nov 2025 16:23:41 GMT
                    Server: uvicorn

                    {"y":76.24499999999999,"u":-37.518150000000006,"v":156.825}
Forms             : {}
Headers           : {[Content-Length, 59], [Content-Type, application/json], [Date, Wed, 26 Nov 2025 16:23:41 GMT],
                    [Server, uvicorn]}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 59
```

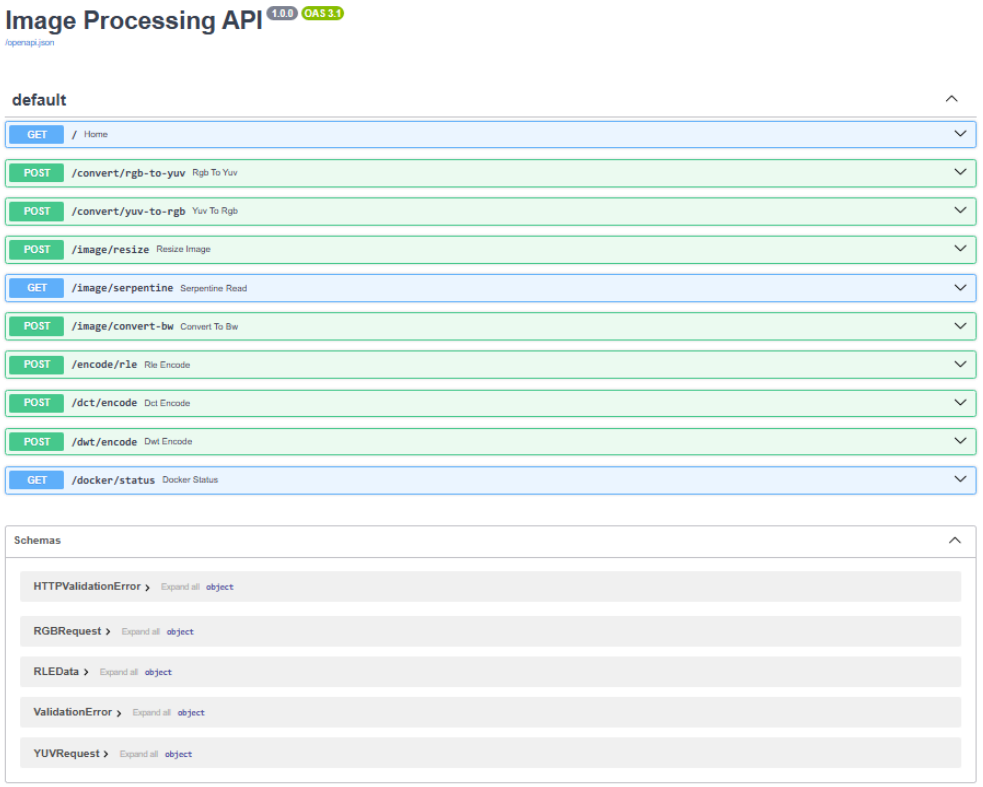
   curl -Method POST http://localhost:8000/encode/rle `
   -Headers @{ "Content-Type"="application/json" } `
   -Body '{"data":"AAAABBBCCDAA"}'

```
PS C:\Users\ngali\OneDrive\Documentos\GitHub\audio-and-video-coding-techniques\practice1> curl -Method POST http://local
host:8000/encode/rle -Headers @{ "Content-Type"="application/json" } -Body '{"data":"AAAABBBCCDAA"}'


StatusCode        : 200
StatusDescription : OK
Content           : {"encoded":[["A",4],["B",3],["C",2],["D",1],["A",2]]}
RawContent        : HTTP/1.1 200 OK
                    Content-Length: 53
                    Content-Type: application/json
                    Date: Wed, 26 Nov 2025 16:32:27 GMT
                    Server: uvicorn

                    {"encoded":[["A",4],["B",3],["C",2],["D",1],["A",2]]}
Forms             : {}
Headers           : {[Content-Length, 53], [Content-Type, application/json], [Date, Wed, 26 Nov 2025 16:32:27 GMT],
                    [Server, uvicorn]}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 53
```

We also find more commands to:
- Test the status of the docker: *curl http://localhost:8000/docker/status*
- Open the API documentation in the browser: *open http://localhost:8000/docs*



5. **Use docker-compose to launch both and make them interact (i.e., you have a method for conversion, you launch your API and it will call the FFMPEG docker).**

   We create an archive *docker-compose.yml* inside the practice1 folder to launch simultaneously the API and the FFMPEG container, and we make the Docker socket to allow the API to query the Docker status by using:
   /var/run/docker.sock and */docker/status*

   By following the FastAPI links above and the following Docker ones:
   https://docs.docker.com/compose/
   https://docs.docker.com/compose/gettingstarted/
   https://docs.docker.com/engine/security/rootless/